SMART CONTRACT SECURITY

Token Audit Report

IMMUNEBYTES

05 MAY 2024

Introduction 3

About Cross The Ages 3

About ImmuneBytes 3

Documentation Details 4

Audit Goals 4

Audit Process & Methodology 5

Audit Details 6

Security Level References 7

Severity Status References 8

Severity Status 8

Finding 9

Critical Severity Issues 10

High Severity Issues 10

Medium severity issues 10

Low severity issues 10

Informational 11

Automated Testing 13

Concluding Remarks 14

Disclaimer 14

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About Cross The Ages

Launched in 2020, and established outside of Web 3, Cross The Ages is first and foremost a multimedia Intellectual Property (IP) grounded in a futuristic fantasy meets sci-fi epic narrative that forms the foundation of an experiential ecosystem blending virtual and real worlds. The CTA ecosystem will include gaming, E-sports, animation, and collectables, as well as an underlying gaming investment model where investment in the gaming virtual World of Artellium can translate into the renewable energy transition through clean energy production in the Real World.

Cross The Ages is a complete and captivating world interconnecting virtual & physical, where free-to-play meets play-and-earn in a universe leveraging blockchain technology and setting new benchmarks for innovation in gaming.

Visit https://crosstheages.com/ to learn more about it.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting security audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 255+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Documentation Details

The team has provided the following doc for audit:

1. https://whitepaper.crosstheages.com/whitepaper/en

Audit Goals

The audit's focus was to verify that the system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within the code and the system.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include
    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

ImmuneBytes team has performed thorough testing of the project, starting with analyzing the code design patterns in which we reviewed the smart contract architecture to ensure it is a structured and safe use of third-party smart contracts and libraries.

Our team then performed a formal line-by-line inspection of the Smart Contract to find potential issues like Signature Replay Attacks, Unchecked External Calls, External Contract Referencing, Variable Shadowing, Race conditions, Transaction-ordering dependence, timestamp dependence, DoS attacks, and others.

In the Unit testing phase, we run unit tests written by the developer to verify that the functions work as intended. In Automated Testing, we tested the Smart Contract with our in-house-developed tools to identify vulnerabilities and security flaws.

The code was audited by a team of independent auditors, which included -

1. Testing the functionality of the Smart Contract to determine proper logic has been followed throughout.
2. Analyzing the complexity of the code by thorough, manual review of the code, line-by-line.
3. Deploying the code on testnet using multiple clients to run live tests.
4. Analyzing failure preparations to check how the Smart Contract performs in case of bugs and vulnerabilities.
5. Checking whether all the libraries used in the code are on the latest version.
6. Analyzing the security of the on-chain data.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

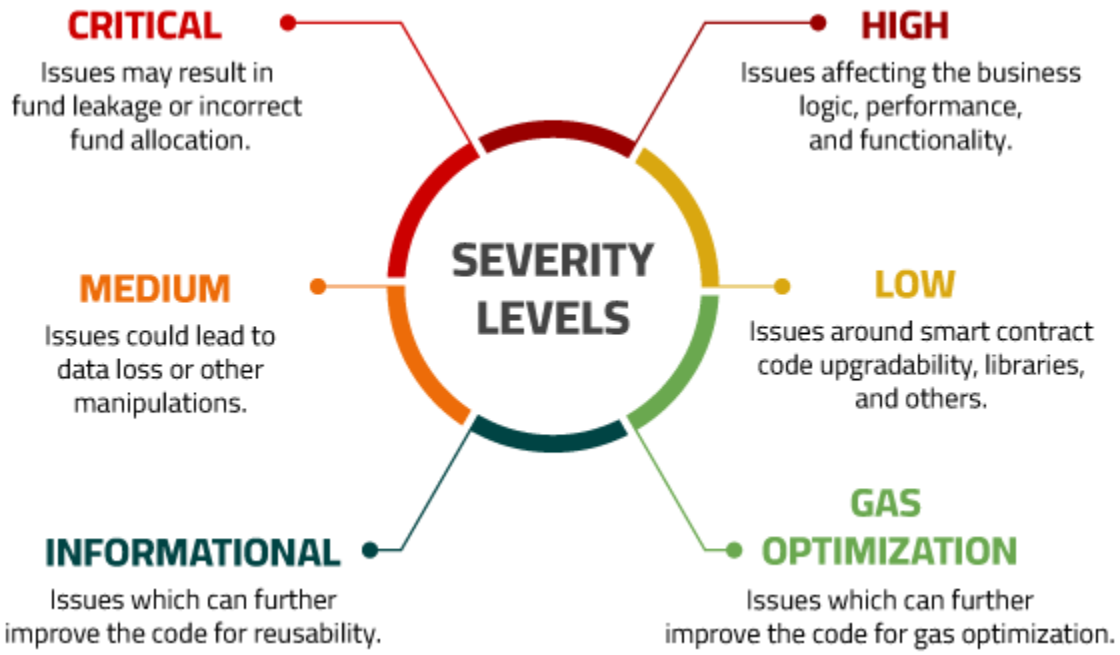
Audit Details

Project Name	Cross The Ages
Platform	EVM
Languages	Solidity
GitHub Link	https://github.com/crosstheages/cta-token/blob/main/contracts
Contracts	CTAToken.sol
Commit	696b4071536acc3ea65c6003cecac46c5991496f
Platforms & Tools	Remix IDE, Truffle, VScode, Contract Library, Slither, SmartCheck, Fuzz
Etherscan Link	0x90685e300a4c4532efcefe91202dfe1dfd572f47

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:



This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Severity Status References

Severity Status

Issues	Critical	High	Medium	Low	Informational	Gas
Open	-	-	-	-	1	-
Fixed	-	-	-	-	2	-
Redacted	-	-	-	-	-	-
Acknowledged	-	-	-	-	-	-

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Finding

#	Finding	Risk	Status
1	Hardcoded address found in contract	**Informational**	**Fixed**
2	Missing NatSpec Annotations in CTAToken Smart Contract	**Informational**	**Fixed**
3	Redundant Declaration of Token Attributes	**Informational**	**Open**

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Critical Severity Issues

No issues were found.

High Severity Issues

No issues were found.

Medium severity issues

No issues were found.

Low severity issues

No issues were found.

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Informational

Issue	Hardcoded address found in contract
Line No.	9
Description	The contract defines constants such as SUPPLY, DECIMALS, and HOLDER. While constants for SUPPLY and DECIMALS are generally acceptable given their unchanging nature in the context of a token, the hardcoding of the HOLDER address is not adequate. Considering the immutable nature of smart contracts, it is not considered a better practice to include hard coded addresses in smart contracts. If the address is compromised, or if there is a need to change the wallet strategy, the token holder's address remains unchanged, posing potential security and operational risks.
Recommendation	Dynamic Initialization: To enhance flexibility and maintain security, it is advisable to initialize critical addresses through constructor parameters. This approach allows the deployment script or the deploying party to specify the holder address at the time of deployment, making the contract adaptable to different environments and requirements. Use of Upgradeable Patterns: If the project anticipates future changes to its logic or parameters, considering an upgradeable contract pattern might be appropriate. Tools like OpenZeppelin's upgrades plugins facilitate this.
Status	**Fixed**

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Issue	Missing NatSpec Annotations in CTAToken Smart Contract
Line No.	-
Description	NatSpec annotations provide valuable documentation directly in the contract source code. They are particularly useful for understanding contract functionalities, auditing, and interfacing with development tools. The CTAToken contract does not include these annotations, missing an opportunity to enhance code clarity and maintainability.
Recommendation	It is recommended to include comprehensive NatSpec annotations throughout the contract.
Status	**Fixed**

Issue	Redundant Declaration of Token Attributes
Line No.	-
Description	The CTAToken contract includes explicit constant state variables NAME and SYMBOL to define the token's name and symbol, respectively. However, these attributes are already managed within the ERC20 standard implementation by OpenZeppelin, which the contract extends. The ERC20 constructor accepts name and symbol as parameters, making these separate constants unnecessary. Implication: Declaring these constants in addition to passing them to the ERC20 constructor results in redundant storage and increased bytecode size, leading to higher deployment and execution costs without providing additional functionality.
Recommendation	The redundant use of state variables can be removed from the contract.
Status	**Open**

This audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program as a complement to this audit is strongly recommended.

Automated Testing

```
Compiled with solc
Total number of contracts in source files: 8
Source lines of code (SLOC) in source files: 177
Number of  assembly lines: 0
Number of optimization issues: 0
Number of informational issues: 6
Number of low issues: 0
Number of medium issues: 0
Number of high issues: 0

ERCs: ERC20

+----------------+-------------+-------+--------------------+--------------+----------+
| Name           | # functions |  ERCS |         ERC20 info | Complex code | Features |
+----------------+-------------+-------+--------------------+--------------+----------+
| IERC721Errors  |           0 |       |                    |           No |          |
| IERC1155Errors |           0 |       |                    |           No |          |
| CTAToken       |          31 | ERC20 |         No Minting |           No |          |
|                |             |       | Approve Race Cond. |              |          |
|                |             |       |                    |              |          |
+----------------+-------------+-------+--------------------+--------------+----------+
```

Concluding Remarks

While conducting the audits of Cross The Ages, our auditors observed that the system contains a few recommendations. Our auditors suggest that the developers resolve issues as a priority before moving on to production. The recommendations will improve the smart contract's operations.

Disclaimer

ImmuneBytes' audit does not provide a security or correctness guarantee of the audited code. Securing protocol is a multistep process; therefore, running a bug bounty program complementing this audit is strongly recommended.

Our team does not endorse the Cross The Ages platform or its product, nor is this audit investment advice.

Notes:

- Please make sure the code deployed on the mainnet is the ones audited.
- Check for code refactoring by the team on critical issues.

IMMUNEBYTES

AUDITS